

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – COMPARATIVE ANALYSIS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	Assessment:	Comparative Analysis
Weightage:	30%	Total Marks:	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	LOKESH S	Reg. No.:	192572151
Section / Batch:	CSE – AI /2 ND YEAR	Date:	25/08/2026

Instructions to Students

- Identify the relevant concepts of L1, L2, and L3 cache, SRAM, DRAM, NAND Flash, MRAM, RRAM, virtual memory, paging, latency, bandwidth, and throughput.
- Analyze the memory requirements of different computer systems by considering cache levels, memory technologies, access latency, bandwidth, throughput, capacity, and energy consumption.
- Evaluate the performance characteristics of different memory technologies and determine their suitability for cache memory, main memory, secondary storage, and future hierarchical memory systems.
- Compute relevant performance measures such as access time, latency, bandwidth, throughput, hit/miss rate, average memory access time (AMAT), speedup, and energy efficiency wherever applicable.
- Interpret the results by assessing memory bottlenecks, performance improvements, technology trade-offs, and the suitability of each memory technology or memory hierarchy for a given application.

QUESTIONS

1. Analyze the relationship between cache size, cache access time, and cache hit rate. Evaluate how changes in cache size affect processor performance, memory latency, and overall system throughput.
2. Assess the impact of memory bandwidth on data-intensive applications such as artificial intelligence, scientific computing, and multimedia processing. Analyze how insufficient bandwidth can create memory bottlenecks and limit processor performance.
3. Evaluate the effect of page size on virtual memory performance. Analyze how page size influences page faults, memory utilization, translation overhead, and system throughput.
4. Examine the role of data locality in improving cache performance. Analyze how temporal and spatial locality influence cache hit rates, memory access latency, and processor efficiency.
5. Assess the suitability of SRAM, DRAM, NAND Flash, MRAM, and RRAM for different levels of a memory hierarchy. Evaluate each technology based on latency, bandwidth, capacity, energy consumption, endurance, and cost, and recommend appropriate technologies for specific applications.

Code of Conduct

I LOKESH S (192572151) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. LOKESH

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Marks Evaluation:

Q. No	Scope of Items (20)	Comparison Depth (25)	Use of Evidence (20)	Critical Thinking (20)	Clarity of Presentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 25	/ 20	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

1. Cache Size, Access Time, and Hit Rate

Cache design sits at the center of processor performance because it bridges the enormous speed gap between the CPU core and main memory (DRAM). Three variables — cache size, access time, and hit rate — are tightly coupled, and design decisions typically trade one against another.

1.1 The Size–Latency–Hit-Rate Relationship

Increasing cache size generally raises the hit rate because a larger cache can retain more of the working set of an application, reducing the number of times the processor must fetch data from slower memory. However, this benefit does not come for free. Larger caches require more address decoding logic, longer wordlines and bitlines, and larger comparator/tag arrays, all of which increase physical propagation delay. As a result, access time grows — often close to logarithmically with capacity for well-designed set-associative caches, though poorly designed large caches can see steeper penalties. This is the classic capacity–latency trade-off that governs why a processor uses multiple cache levels (L1, L2, L3) rather than one large cache.

1.2 Diminishing Returns and the Working-Set Effect

Hit rate does not scale linearly with size. For a given workload, there exists a working set — the set of memory locations actively reused within a short time window. Once the cache is large enough to hold this working set, further capacity increases yield diminishing improvements in hit rate, following a curve that is steep at small sizes and flattens at larger sizes. Beyond this point, additional area is better spent on lower-level caches, prefetchers, or core resources rather than continued enlargement of a single cache level.

1.3 Effect on Processor Performance

- Higher hit rate reduces the effective memory access time (EMAT), computed as $EMAT = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$, directly shortening the average cycles per instruction (CPI).
- Lower access time at a fixed hit rate allows a higher clock frequency for the cache pipeline stage, improving instruction throughput.
- A poorly sized cache — too small — causes frequent capacity misses and stalls the pipeline; too large a single-level cache can slow every access, even hits, because of increased physical delay.
- Multi-level hierarchies resolve this tension: a small, fast L1 minimizes typical-case latency, while progressively larger L2/L3 caches absorb L1 misses without penalizing every access with worst-case latency.

1.4 Effect on Memory Latency and System Throughput

From the memory system's perspective, a higher cache hit rate reduces the number of requests that reach DRAM, which lowers effective memory latency as experienced by the processor and reduces contention on the memory bus. This has a compounding effect in multi-core systems: fewer last-level cache misses mean less pressure on shared memory bandwidth, which in turn reduces queuing delay for all cores.

2. Memory Bandwidth and Data-Intensive Applications

Memory bandwidth — the rate at which data can be transferred between memory and the processor — has become a first-order constraint for modern workloads, often more limiting than raw compute throughput. This section evaluates its impact across three representative domains.

2.1 Artificial Intelligence and Machine Learning

Deep learning training and inference are dominated by large matrix and tensor operations that stream weights, activations, and gradients between memory and compute units. Modern accelerators (GPUs, TPUs) have compute throughput measured in hundreds of teraflops, but many workloads — particularly inference with small batch sizes and memory-bound operators such as attention mechanisms in large language models — cannot keep the arithmetic units fed. This condition, where achievable performance is capped by data movement rather than compute capability, is described by the roofline model: an operation's arithmetic intensity (operations per byte moved) determines whether it is compute-bound or memory-bound. Because many AI workloads have low arithmetic intensity, high-bandwidth memory (HBM) has become essential in accelerator design, often providing several times the bandwidth of conventional DDR memory.

2.2 Scientific and High-Performance Computing

Scientific simulations — computational fluid dynamics, climate modeling, molecular dynamics, and sparse linear algebra — frequently exhibit irregular memory access patterns and low reuse, making them acutely bandwidth-sensitive. Sparse matrix operations, for example, touch large data structures with limited data reuse per byte fetched, so performance tracks memory bandwidth far more closely than peak floating-point rate. This is why supercomputer architectures increasingly prioritize aggregate memory bandwidth and interconnect bandwidth alongside FLOPS, and why benchmarks such as STREAM (which measures sustained memory bandwidth) are considered as informative as peak-FLOPS benchmarks for HPC procurement.

2.3 Multimedia and Signal Processing

Video encoding/decoding, image processing, and real-time rendering involve continuous streaming of large frame buffers and texture data. These workloads combine moderate compute intensity with very high data volume — a single uncompressed 4K video frame is tens of megabytes, and pipelines must move many such frames per second. Insufficient bandwidth manifests as dropped frames, increased latency, or reduced resolution/quality settings, which is why GPUs dedicate substantial die area and power budget to wide memory buses and specialized memory technologies such as GDDR.

2.4 Bottleneck Formation and Its Effect on Processor Performance

When the rate of data demand from compute units exceeds available bandwidth, request queues at the memory controller grow, average latency increases, and functional units stall waiting for operands — a condition often called the memory wall. The effect compounds with core count: as more cores or execution units are added without proportional bandwidth growth, per-core effective bandwidth shrinks

3. Page Size and Virtual Memory Performance

Virtual memory maps address spaces into fixed-size pages, and the chosen page size has broad, sometimes conflicting, effects on system performance.

3.1 Page Faults and Memory Utilization

- Smaller pages (e.g., 4 KB) allow finer-grained allocation, reducing internal fragmentation — the wasted space when a process's data does not fill an entire page — which improves effective memory utilization, especially for many small processes.
- Smaller pages can increase the number of page faults for large, sparsely accessed working sets, since more distinct page-table entries are needed to cover the same address range and each entry maps a smaller amount of data.
- Larger pages (2 MB, 1 GB "huge pages") reduce the number of pages needed to cover a large working set, lowering page-fault frequency for workloads with large, contiguous, or predictable access patterns (e.g., in-memory databases, large ML models).
- Larger pages increase internal fragmentation risk, since even a few bytes of use commit an entire large page, potentially wasting substantial memory when allocations are small or irregular.

3.2 Translation Overhead and the TLB

Every virtual-to-physical address translation is a candidate for caching in the Translation Lookaside Buffer (TLB). A TLB entry maps one page, so larger pages let a fixed-size TLB cover a much larger address range, dramatically reducing TLB miss rates for large working sets. TLB misses are costly because they trigger a page-table walk — potentially several sequential memory accesses — so reducing miss frequency through larger pages can measurably lower average memory access latency for workloads like large databases, virtualization hosts, and big-data analytics. Conversely, small pages increase the likelihood of TLB misses when the working set spans many pages, adding translation overhead on top of ordinary data access latency.

3.3 System Throughput Implications

The net throughput effect of page size depends on workload characteristics. Applications with large, regular working sets (scientific computing, databases, virtual machines) benefit from huge pages through fewer TLB misses and fewer page-table walks, often yielding measurable throughput gains with minimal downside. Applications with many small, short-lived, or sparse allocations (interactive desktop workloads, containers with tight memory limits) are better served by smaller pages, since huge pages would waste physical memory and could increase page-fault service time when a huge page must be prepared or reclaimed. Modern operating systems mitigate this trade-off through transparent huge pages and multi-size page support, dynamically choosing page granularity based on allocation behavior.

4. Data Locality and Cache Performance

Locality of reference is the empirical tendency of programs to access a disproportionately small set of memory locations repeatedly and to access nearby locations close together in time. Cache design fundamentally exploits two forms of locality.

4.1 Temporal Locality

Temporal locality is the tendency to re-reference the same memory location within a short time period — for example, a loop counter or an accumulator variable accessed on every iteration. Caches exploit temporal locality by retaining recently accessed data, so a second reference to the same address can be served from the fast cache rather than from slower memory. Workloads with strong temporal locality (tight loops, frequently called small functions, repeatedly accessed lookup tables) achieve high hit rates even in relatively small caches, because the same small set of addresses is reused constantly.

4.2 Spatial Locality

Spatial locality is the tendency to reference memory locations near ones recently accessed — for example, iterating sequentially through an array. Caches exploit this by fetching data in blocks (cache lines) larger than a single word, so that accessing one element pulls neighboring elements into the cache preemptively. Sequential or predictable stride access patterns benefit strongly from spatial locality, while random or pointer-chasing access patterns (e.g., traversing linked lists or hash tables with poor memory layout) exhibit weak spatial locality and suffer higher miss rates regardless of cache size.

4.3 Combined Effect on Hit Rate, Latency, and Processor Efficiency

Programs that exhibit both strong temporal and spatial locality achieve the highest cache hit rates because nearly every access is satisfied by data already resident in the cache from a previous access to the same or a nearby address. This directly reduces average memory access latency, since hits are serviced in a few processor cycles compared to tens or hundreds of cycles for a main-memory access. Improved hit rates cascade into higher processor efficiency: fewer pipeline stalls, better utilization of out-of-order execution resources, and more effective prefetching, since hardware prefetchers rely on detecting spatial patterns to speculatively load data ahead of demand. Software techniques such as loop tiling/blocking, data structure reorganization (structure-of-arrays instead of array-of-structures), and cache-conscious algorithm design exist specifically to strengthen locality and thereby improve cache performance without changing the underlying hardware.

5. Memory Technologies Across the Hierarchy

Different memory technologies present distinct trade-offs among latency, bandwidth, capacity, energy consumption, endurance, and cost. Matching technology to hierarchy level is central to balanced system design.

5.1 Comparative Technology Assessment

Technology	Latency	Bandwidth	Capacity/Density	Energy	Endurance	Relative Cost/bit
SRAM	Sub-ns to ~1 ns (fastest)	Very high	Low (6T cell, large area)	Low per access, but high static leakage	Effectively unlimited	Highest
DRAM	~10–50 ns	High	High (moderate density)	Moderate; requires periodic refresh	Effectively unlimited	Moderate
NAND Flash	~10–100 μ s (read); ms (write/erase)	Low–moderate	Very high	Low idle power; no refresh needed	Limited (~1,000–100,000 program/erase cycles)	Low
MRAM	~10–30 ns (comparable to DRAM/SRAM)	Moderate–high	Moderate	Low; non-volatile, no refresh	Very high (near-unlimited)	High (currently)
RRAM	~10–50 ns	Moderate	High (potential for 3D stacking)	Low; non-volatile	Moderate (better than NAND, less than MRAM)	Moderate (emerging)

5.2 Suitability by Hierarchy Level

SRAM: Its sub-nanosecond access time and very high bandwidth make it the only practical choice for L1 and L2 caches directly integrated with the processor core, where every cycle of latency directly affects instruction throughput. Its high cost per bit and large cell area (typically six transistors) limit it to small capacities, which is why SRAM is never used for main memory despite its speed advantage.

DRAM: The balance of moderate latency, high bandwidth, and much lower cost per bit than SRAM makes DRAM the standard choice for main memory. It offers sufficient capacity for modern operating systems and applications while remaining fast enough to service last-level cache misses without excessive stalling, though it requires periodic refresh cycles that consume power and introduce minor access conflicts.

NAND Flash: Its high density and low cost per bit, combined with non-volatility, make NAND Flash the dominant technology for solid-state secondary storage (SSDs) and mobile/embedded storage. Its microsecond-to-millisecond latency and limited write endurance make it unsuitable for main memory or cache roles, but excellent for persistent bulk storage where access patterns tolerate higher latency.

MRAM: With latency approaching DRAM/SRAM and near-unlimited write endurance combined with non-volatility, MRAM is well suited to niche roles such as embedded non-volatile memory, cache in energy-constrained or radiation-hardened systems (aerospace, industrial), and potential last-level cache or persistent-memory applications where data must survive power loss. Its current cost and density disadvantages limit it from replacing DRAM at scale today.

RRAM: Offering a favorable combination of speed, density potential (including 3D crossbar stacking), and non-volatility, RRAM is a promising candidate for storage-class memory — a tier bridging the latency/cost gap between DRAM and NAND Flash — as well as for emerging applications such as in-memory computing and neuromorphic/AI accelerators, where its analog conductance states can represent synaptic weights directly.

5.3 Application-Oriented Recommendations

- General-purpose processors (desktop/server CPUs): SRAM for L1–L3 cache; DRAM for main memory; NAND Flash (SSD) for persistent storage — the conventional, cost-effective hierarchy.
- AI training/inference accelerators: SRAM for on-chip scratchpad/cache; high-bandwidth DRAM variants (HBM) for model and activation storage, given the extreme bandwidth demands identified in Section 2.
- Embedded and IoT systems with intermittent power: MRAM or RRAM for working memory, since non-volatility preserves state across power interruptions without the energy and complexity cost of DRAM refresh.
- Storage-class memory / persistent memory tiers: RRAM or MRAM positioned between DRAM and NAND Flash to reduce the latency gap for large in-memory databases and fast checkpointing, particularly where NAND endurance would be a limiting factor.
- Mass storage and archival systems: NAND Flash (or, for the coldest data, higher-latency technologies not covered here) remains the economical choice given its low cost per bit and adequate performance for sequential or infrequent access.